# Week 11: DOE & Power Analysis

November 4, 2026

## THE POINT OF THIS WEEK

So far the data came to us and we estimated whatever the data could support. This week we flip it: **the design of the survey decides what you are able to learn from it.** Two questions, both of which you have to answer before fielding the final survey:

1. **Which questions should I ask?** A badly chosen set of profiles makes some effects impossible to separate from each other, no matter how many people you survey. That is the *design of experiment* (DOE) problem.
2. **How many people do I need?** Precision costs respondents, and respondents cost money. That is the *power analysis* problem.

The answers are connected: a more efficient design reaches the same precision with fewer respondents.

## MAIN & INTERACTION EFFECTS

For three two-level attributes A, B, and C there are 8 possible profiles — the corners of a cube. Label a profile by which attributes are "on": `I` is all off, `AB` is A and B on and C off, `ABC` is all three on.

A **main effect** is the average change in the outcome when one attribute goes from off to on, averaged over everything else:

$$ME(a) = \left(\frac{A + AB + AC + ABC}{4}\right) - \left(\frac{I + B + C + BC}{4}\right)$$

An **interaction effect** is the change in one attribute's effect depending on the level of another:

$$INT(ab) = \frac{1}{2}\left[\left(\frac{AB + ABC}{2}\right) - \left(\frac{B + BC}{2}\right)\right] - \frac{1}{2}\left[\left(\frac{A + AC}{2}\right) - \left(\frac{I + C}{2}\right)\right]$$

### Wine pairings

Rate four meals on a 1-10 scale: steak-red 8, steak-white 5, fish-red 7, fish-white 9.

| Effect | Computation | Result |
|---|---|---|
| ME wine (white to red) | mean(red) − mean(white) | +0.5 |
| ME meat (fish to steak) | mean(steak) − mean(fish) | −1.5 |
| INT meat × wine | ((8−7) − (5−9)) / 2 | +2.5 |

The main effects alone say "people slightly prefer red, and prefer fish." The interaction says something much more useful: **red with steak, white with fish**. A model with main effects only would have missed it entirely.

Plot interactions as one line per group: non-parallel lines mean an interaction.

```
ggplot(data) +
  geom_line(aes(x = wine, y = rating,
                group = meat, color = meat))
```

## BALANCE, ORTHOGONALITY, CONFOUNDING

| Property | Means |
|---|---|
| **Balanced** | Every level of an attribute appears an equal number of times |
| **Orthogonal** | Every *pair* of levels appears together an equal number of times |
| **Confounded** | Two effects move together, so the data cannot tell them apart |

A **full factorial** design uses every possible profile. It is always balanced and orthogonal, so nothing is confounded — but with real attributes it is far too big to ask anyone. A **fractional factorial** design uses a subset, and *which* subset you take determines what gets confounded.

```
profiles <- data.frame(cbc_profiles(
  electric = c(1, 0),
  warranty = c(1, 0),
  ford     = c(1, 0)))

profiles %>% count(electric)   # balance

profiles %>%                   # orthogonality
  mutate(ew = electric == warranty) %>%
  count(ew)
```

Two different quarter-fractions of the same 8 profiles:

| Rows kept | Result |
|---|---|
| 1, 4, 5, 8 | Balanced, but `electric` and `warranty` are confounded |
| 1, 4, 6, 7 | Balanced, no main effects confounded — each main effect is confounded with a two-way interaction instead |

That second design is the usable one: you can estimate all three main effects, you just cannot also estimate interactions. **Confounding never disappears, it only moves.** Good design puts it where you do not care.

### Finding what is confounded with what

If a design forces A to equal −B, multiply that relation through every other effect to find the rest of the confounding pattern:

| Multiply by | Gives |
|---|---|
| b | $ab = -I$ |
| c | $ac = -bc$ |
| ab | $b = -a$ |
| ac | $c = -abc$ |

## DESIGN EFFICIENCY

Balance and orthogonality are rules of thumb. The actual measure of how much a design teaches you is the **information matrix**: the negative hessian of the log-likelihood, evaluated at an assumed set of coefficients.

$$\boldsymbol{I}(\beta) = -\nabla^2_\beta \ln(\mathcal{L})$$

A **D-optimal** design minimizes **D-error**, where $p$ is the number of coefficients in the model:

$$D = |\ \boldsymbol{I}(\beta)\ |^{-1/p}$$

More information, smaller D-error, smaller standard errors — the same curvature argument as last week, now applied *before* you have any data.

Because D-error depends on $\beta$, a **Bayesian D-efficient** design needs **priors**: your assumption about the coefficients. The design is efficient *for those priors*.

```
priors <- cbc_priors(
  profiles,
  price       = -0.7,
  fuelEconomy = 0.1,
  accelTime   = -0.2,
  powertrain  = c('Hybrid' = 0.5,
                  'Electric' = -4.0))

design <- cbc_design(
  profiles = profiles,
  n_resp   = 500,  # respondents
  n_alts   = 3,    # alternatives per question
  n_q      = 6,    # questions per respondent
  priors   = priors,
  method   = 'modfed',
  n_start  = 1,
  n_blocks = 1)

cbc_inspect(design)
```

### The seven `method` options

| Method | What it does |
|---|---|
| `"random"` | Default. Samples profiles independently per respondent. No priors |
| `"shortcut"`, `"minoverlap"`, `"balanced"` | Fast greedy algorithms targeting balance and low overlap. No priors |
| `"stochastic"` | D-optimal; takes the first improvement. Fastest of the three |
| `"modfed"` | D-optimal; modified Federov, swaps whole profiles |
| `"cea"` | D-optimal; coordinate exchange, swaps one attribute at a time |

Give the D-optimal methods priors and they minimize $D_p$-error; omit priors and they minimize $D_0$-error, which assumes every coefficient is zero. `n_start` is the number of random starts (a better search, but slower); `n_blocks` splits the design so more people see more different questions.

### Comparing designs

```
cbc_compare(
  Random     = design_rand,
  Stochastic = design_stochastic,
  Modfed     = design_modfed,
  CEA        = design_cea,
  metrics = c("efficiency", "balance", "overlap"))
```

Three things that table will show you:

- **The D-optimal methods disagree with each other.** Same goal, different searches, different answers. Try more than one.
- **The random design reports `NA` for D-error.** {cbcTools} computes D-error *during* the optimization, so only designs built with a D-optimal method have one. There is no function that scores an arbitrary design after the fact.
- **The random design is often the *most* balanced.** Not a bug. A D-optimal algorithm will trade balance away to buy information about the coefficients you told it to care about.

> Balance and efficiency are different goals, and D-error only exists for D-optimally built designs. The measure that compares **any** two designs — and the only one denominated in something you can budget for — is power.

## POWER ANALYSIS

Standard errors shrink with the square root of the sample size:

$$se \propto 1/\sqrt{N}$$

So halving your standard errors takes **four times** the respondents. Standard errors also shrink with fewer attributes, fewer levels per categorical attribute, and more questions per respondent.

The simulation-based answer: build a design, simulate choices from your priors, then re-estimate the model on larger and larger subsets of respondents.

```
choices <- cbc_choices(design = design,
                       priors = priors)

power <- cbc_power(
  data     = choices,
  outcome  = "choice",
  obsID    = "obsID",
  n_q      = 6,
  n_breaks = 10,
  pars = c("price", "fuelEconomy",
           "accelTime", "powertrain"),
  n_cores = 1)

plot(power, type = "power")
plot(power, type = "se")
summary(power)
```

`summary()` answers the actual question: the sample size at which each coefficient reaches 80% power at $\alpha = 0.05$.

`pars` takes **attribute** names, not dummy-coded column names — `cbc_power()` dummy codes for you and reports `powertrainHybrid` and `powertrainElectric` separately. Leave `pars` out and it auto-detects every attribute. This is unlike calling `logitr()` directly, where you dummy code first.

### Reading the output honestly

A power analysis is only as good as the priors you feed it. With large assumed effects every coefficient clears 80% power at the smallest sample you tested, and the power curves tell you nothing. **Always run a conservative set of priors too**, and report which priors your sample size came from.

```
priors_conservative <- cbc_priors(
  profiles,
  price       = -0.1,
  fuelEconomy = 0.1,
  accelTime   = -0.1,
  powertrain  = c('Hybrid' = 0.1,
                  'Electric' = -0.4))
```

When power has maxed out, the `type = "se"` plot is the informative one — standard errors keep shrinking after power has flattened.

Compare designs in power units:

```
plot_compare_power(
  Random = power_rand,
  Modfed = power_modfed,
  type = "power")
```

The D-efficient design reaches any given precision with fewer respondents. Same conclusion as the D-error comparison, now in units you can put in a budget.

### The {cbcTools} pipeline

| Function | Does |
|---|---|
| `cbc_profiles()` | All combinations of attributes and levels |
| `cbc_priors()` | Assumed coefficients |
| `cbc_design()` | Build the design of experiment |
| `cbc_inspect()` | Balance, overlap, D-error of one design |
| `cbc_compare()` | Metrics for several designs side by side |
| `cbc_choices()` | Simulate choices from priors |
| `cbc_power()` | Re-estimate over increasing sample sizes |
| `summary(power)` | Sample size needed per coefficient |
| `plot_compare_power()` | Power or SE curves for several designs |

## BEFORE NEXT CLASS

- HW 10: WTP & Simulations — due **Nov 9**.
- Final Survey — due **Nov 15**. Use this week's power analysis to justify the sample size, and the design comparison to justify the design you field.
- Refine your attributes and levels while you still can. Dropping a level or an attribute is one of the cheapest ways to shrink the sample size you need.

You should be able to:

- Compute a main effect and an interaction effect from a full factorial design.
- Check whether a design is balanced and orthogonal, in R.
- Say what is confounded with what in a fractional factorial design.
- Explain what D-error measures and why it needs priors.
- Build a Bayesian D-efficient design with `cbc_design()` and compare methods with `cbc_compare()`.
- Run `cbc_power()` and report the sample size needed for 80% power.
- Explain why your reported sample size depends on the priors you assumed.